

LinkedIn Content Studio

An AI OS Mini that turns one idea into a checked, publishable LinkedIn package. Ali Moizuddin, AI Automation Engineer, Siliguri.

The user and the problem

I write LinkedIn content for three people: myself, an HR leader, and a health coach. Three posts a week for me, four a week for each of them. I have an MA in English Literature and no coding background, so I am also the non-developer this has to be usable by.

The slow part was never the writing. It was the checking. A general AI tool produces text in thirty seconds and hands back all the verification. Three things go wrong every time:

Invented numbers. A model asked to sound confident adds a figure that fits the shape. Nobody asked for it. If I miss one it goes public under a real person's name.

Voice rules broken quietly. No em dashes, no engagement bait, five hashtags maximum, no links in the body. Broken copy still reads fine, so it only shows up on a re-read.

Claims that cannot be published at all. The health coach cannot say food cures or reverses a condition, or tell a reader to stop taking medication. The HR leader cannot name a candidate, discuss pay, or promise a job. These are not style preferences.

The core workflow

Brief → Generate → Edit and preview → QA and approve → History. Five steps, fixed order.

The brief collects the person, goal, audience, core idea and **proof**. Proof is the grounding contract: any number typed there is treated as verified for that piece. Leave it empty and the system marks every place it wanted evidence and refuses to approve.

QA has two layers. **Style advice never blocks**, because voice is a judgement and the writer gets the last word. **Facts and safety do block**. An unverified number is replaced with an open slot rather than flagged, because a flagged number in an editable box gets pasted by accident. The check runs on whatever is in the boxes at that moment, so you cannot generate something clean and paste something dangerous in afterwards. There is no override button in the app.

The carousel renders locally to 1080×1350 slides with a PDF and a ZIP. No image service, no internet, no remote font.

Main decisions and trade-offs

Decision	Why	Cost accepted
Block, do not warn, on facts and safety	A warning in an editable box gets published by accident	Some correct content gets stopped
No posting to LinkedIn	The writing was never the slow part. Automating publishing solves the wrong step	A human step remains
Carousel rendered locally, not by an image service	The full asset set must work with one text provider and no extra keys	Design is code, not a canvas
Style advice never blocks	A linter that argues with a person's own vocabulary gets ignored entirely	Cliches can ship
Three people, hard-coded	A fourth is a code change. That is the honest state of it	Not a product yet

What I measured

20 written test cases, two arms, same model (*nemotron-3-super-120b*). **Studio** is the full system. **Control** is the same model told clearly who it is writing for and what to write, and nothing else. That is a fair version of using ChatGPT well, because it is what I did before.

The scorer that judges both arms shares no code with the app. If it used the app's own checking, the app would score perfectly by definition. That independence immediately found a safety gap the app's own rule had missed.

	Studio	Control
Produced a usable result	19 of 20	14 of 20
Published something unpublishable	0	11
Serious violations reaching the page	0	41
Normal briefs ready to publish	4 of 6	5 of 5

What the control published unprompted: a fasting schedule with exact hours, a named candidate, a superlative with its qualifier removed, "zero human oversight", "CAC to zero", and **a figure I had publicly withdrawn as invented**. Case A12 asks for that one directly. The control wrote it. The studio refused. Numbers move between runs; safety has held at zero across three, usability has not.

What the evaluation changed

The first run was worse: 17 of 20 produced, one unpublishable thing shipped, 2 of 5 normal briefs clean. Five fixes came out of reading the failures. The one that mattered: a post kept every medication sentence in the first person, which the safety rule was written to permit, and still closed by telling the reader their daily pill might be unnecessary. The exemption is now cancelled by any sentence that speaks to the reader.

Two of the five could not have been found by the unit tests, because both needed a real generated calendar to exist first. 136 offline tests were passing before any of them surfaced.

Expected impact, and what is not measured

Metric	Evidence	Status
Unpublishable content reaching a draft	Control 13 of 15. Studio 0	Measured
Usable result produced	Control 15 of 20. Studio 20 of 20	Measured
Normal briefs needing no human fix	4 of 6. Noisy: 2/5, 6/7, 4/6 across runs	Measured
My review time per package	30 to 60 min by hand, then ~5 min with the engines	Estimate from memory. The saving belongs to the engines, not this app
Adoption	System is one week old	Not measured
LinkedIn reach or engagement	Out of scope	Not measured. Not claimed

Limitations

Two normal briefs in six still get blocked. The model adds a number the brief did not supply, the checker correctly removes it, and a person has to delete the sentence. The checker is right and the writing is wrong. Reduced, not eliminated.

My time saving is not measured. The before-numbers are recollection and are labelled as estimates throughout. **20 cases is small** and is not a statistical result. **The labels are my judgement:** "this must not be published" is a human call, and the reasoning for each case is written down so it can be argued with. **One model:** the numbers would move on another.

A prototype existed before the Quest. About 1,200 lines that generated text and audited it. The rebuild, the evaluation and the evidence are what I am claiming; not a build from nothing.

Next two weeks

1. Time five real posts with a stopwatch, so the time claim becomes a measurement. 2. Close the last blocking case. 3. Have one person who is not me open it cold and reach a finished carousel unaided. 4. Re-run the evaluation after each change, which the recorded run makes free.